



DATA STRUCTURES

CONTENTS



13. Applications of Queue ADT

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept of an important ADT – the Queue ADT. In this module we will discuss some important applications of Queues.

Learning Objectives

The learning objectives of the introductory module are as follows:

- To discuss the different applications of Queue
- To understand the use of queue for printing jobs & customer service
- To explain the simulation of Breadth First Search using Queues
- To discuss the use of queues for encoding messages

13.1 Applications of Queue ADT

Queues are basically used in applications which require some kind of servicing in

the order of arrival. Some typical scenarios include

- Customer servicing for any type application for example waiting for Printer service
- Operating Systems where processes need memory to be allocated in the order of arrival
- Round Robin scheduler where jobs are scheduled again in the order of arrival in a round robin manner

Queues are also used as buffer where the order of processing is dictated by First in First out (FIFO) manner. Some examples in this category include:

- Store the vertices yet to be processed in the case of Breadth First Search
- Storing characters for encoding messages

The most common application of queues is in implementing **client-server models**.

For example multiple clients may be requesting services from one or more servers. In this situation some clients may have to wait while the servers are busy. These waiting clients are placed in a queue and serviced in the order of arrival. These type of situations occurs in our everyday life also such as in grocery stores, banks, & airport queues.

13.2 Printing Job Management

Another example that we will use to illustrate the use of queues is in the management of printer jobs. Many users send their printing jobs to a public printer. The printer will put them into a queue according to the arrival time and print the jobs one by one. Figure 13.1 shows the processing of printer jobs. Let us assume that the documents are A.doc, B.doc, C.doc first arrive for printing. Therefore the three jobs are enqueued and A.doc is sent for printing. Next A.doc finishes and is dequeued. Therefore B.doc starts printing. Meanwhile D.doc arrives and is enqueued. while B.doc is still printing. Next B.doc finishes and is dequeued. Now C.doc the document at the front of the queue is sent for printing. Next C.doc

finishes and is dequeued. Now the final item in the queue, D.doc is sent for printing. This process will continue as long as documents arrive for printing.

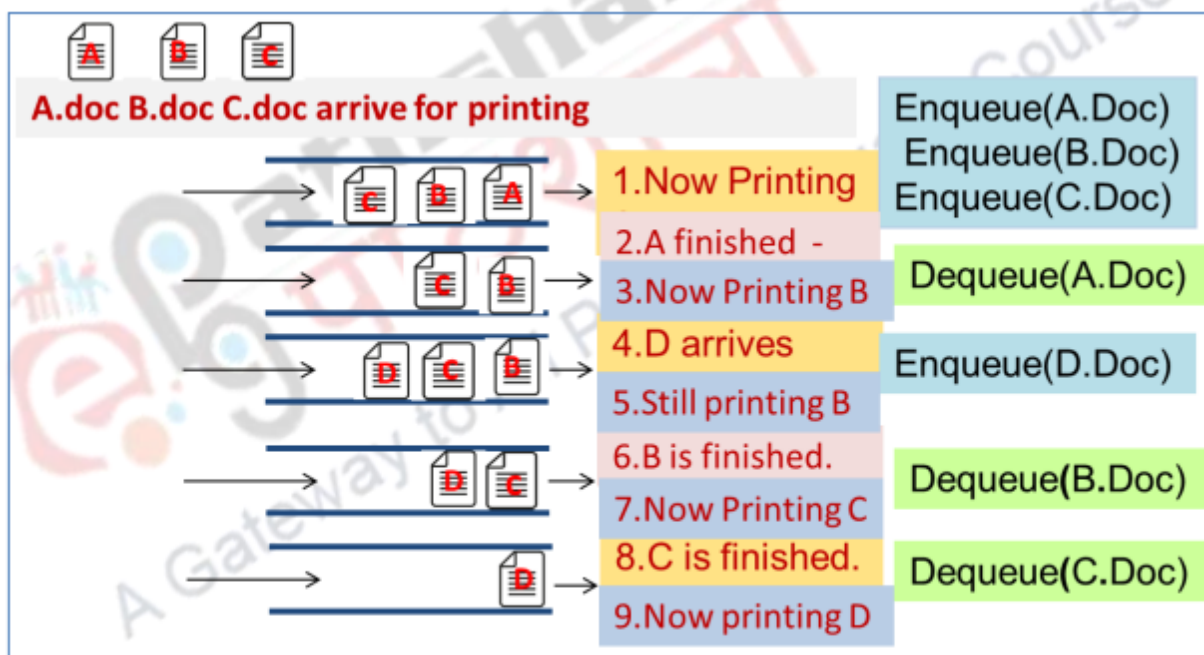


Figure 13.1 Printer Queue

13.3 Customer Service at State Bank of India

Let us consider another example that of customer service in the State Bank of India. Let us assume that every 3 minutes, a new customer arrives at end of the waiting line. Now let us assume that each customer will need 5 minutes for the service. We want to print out the following information after the first 30 minutes that is the time of arrival and departure of each customer, the number of customers in the line, and the identity of the customer currently being served.

For this purpose create a new queue and initialize the following parameters time, incomingcust and servicetime to 0. We will be servicing customers one by one from the queue and putting customers into the queue as they arrive. We need to print the the status after 30 Minutes. The basic steps is given in Figure 13.2.

While time ≤ 30

– If Queue not empty – one customer service is working (use **IsEmpty** to check)
and

- Increment servicetime
- Assume service time is 5 minutes
- **Dequeue** the customer &
- Print deleted customer name & time of finishing
- Start servicetime for next customer

Every 3 minutes a customer arrives & we need to insert customer to the queue
Increment incomingcust

Enqueue Customer

Print Customer Name & time of Entry (**QueueRear**)

Increment Time

Print the status after 30 minutes

If **QueueSize()** $\neq 0$

Size of Queue indicates number of customers waiting

QueueFront() – Indicates current serving Customer

Else No waiting customers

Figure 13.2 Application of Queues for Customer Service

Queue ADT for Customer Service Queue needs the following operations

- CreateQueue
- Enqueue
- Dequeue
- IsEmpty
- QueueFront
- QueueRear
- QueueSize

13.3 Round-Robin Scheduler

Round robin scheduler can be naturally implemented using a queue, Q, by repeatedly performing the following steps:

- $e = Q.dequeue()$; Service element e ; $Q.enqueue(e)$

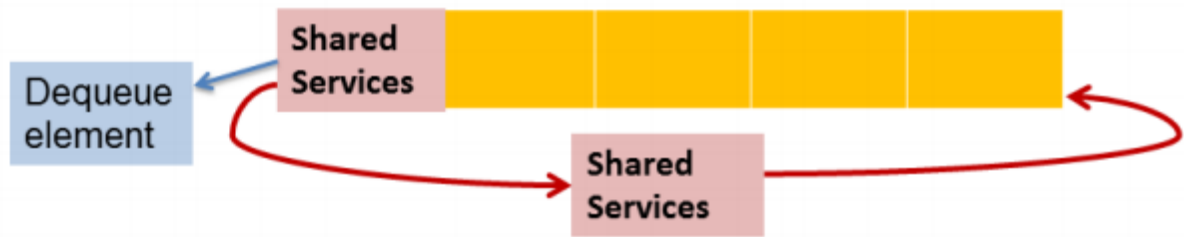


Figure 13.3 Application of Queues for Round Robin Processing

13.5 Queue Application – Breadth First Search

Let us first discuss Breadth First Search (BFS) algorithm and see how queues are used to implement this algorithm.

13.5.1 Breadth First Search (BFS) Algorithm

Breadth-first search (BFS) is a general technique for traversing a graph. One starts at some arbitrary node as the start node and expand shallowest unexpanded node before exploring deeper levels. The steps of the algorithm are explained in Figure 13.4. The algorithm uses a queue to keep track of nodes to visit where new successors go to the end of the queue. Each entry in the queue stores all edges of the node processed and new nodes are added to the rear of the queue.

Step 1: Initially all nodes are undiscovered. Mark the start node as discovered Enqueue the start node S Front of queue which initially is the start node S . Now enqueue all neighbours of node S into the Queue*/

Step2: Repeat Until queue is Empty

a Select the node F in front of queue & mark it as discovered and process it by examining its neighbours.

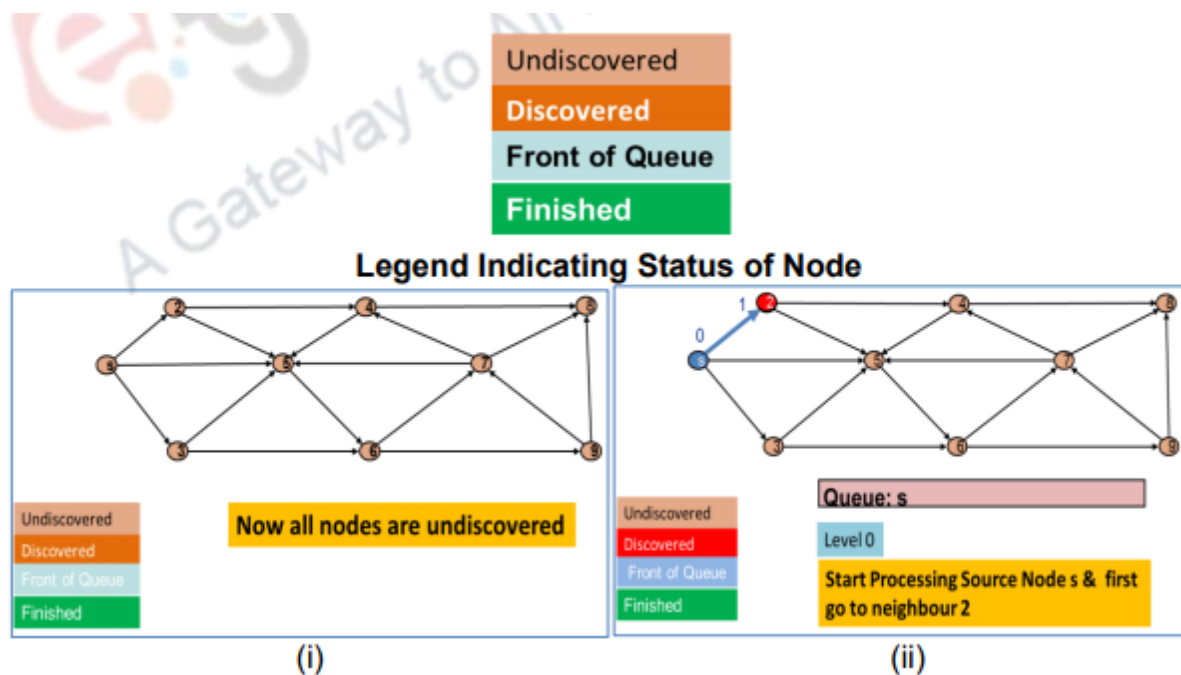
b Repeat until all neighbours of F have been processed

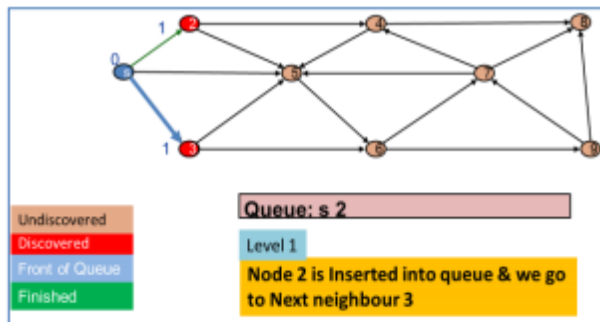
If a neighbour of F has not yet been discovered, add it to the rear of queue

else do not add it to queue

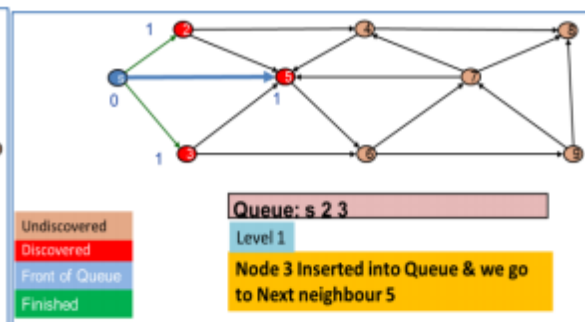
c Now delete the original node from the queue and go to step 2

Figure 13.4 Steps of the Breadth First Algorithm Using Queues

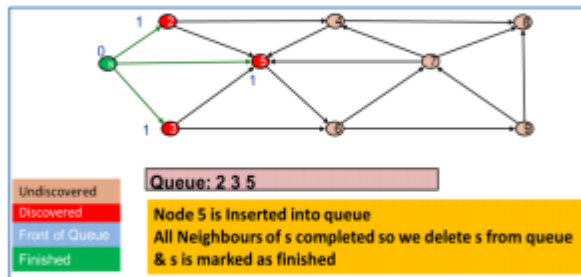




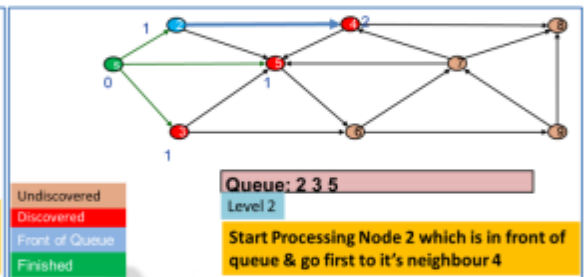
(iii)



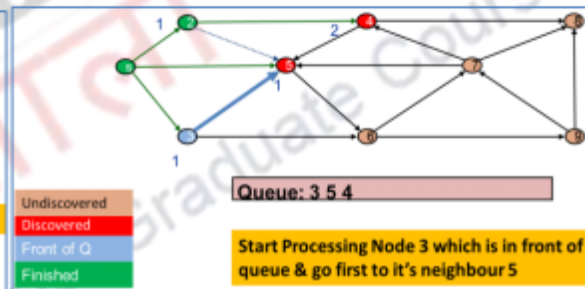
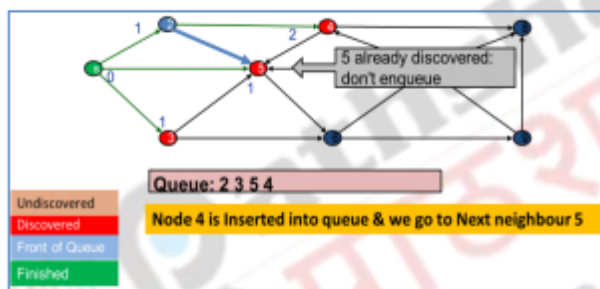
(iv)

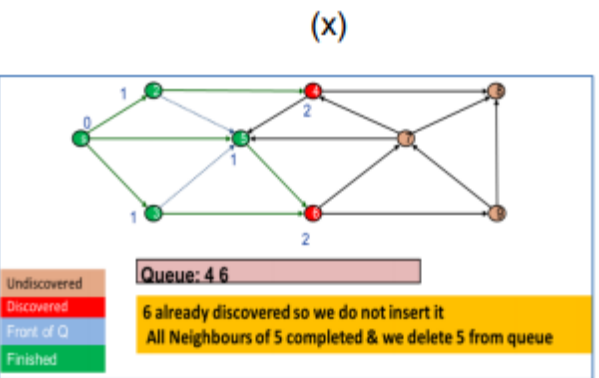
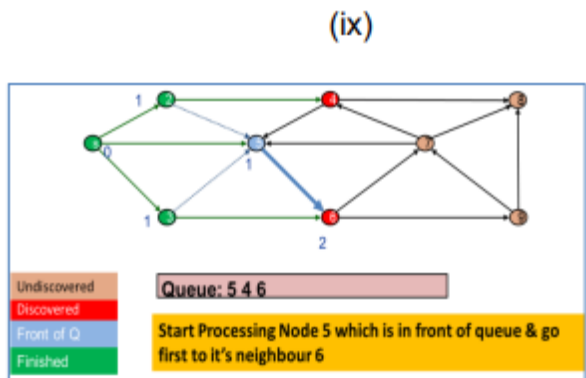
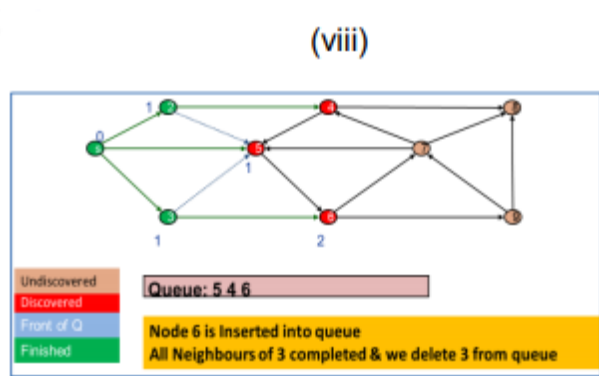
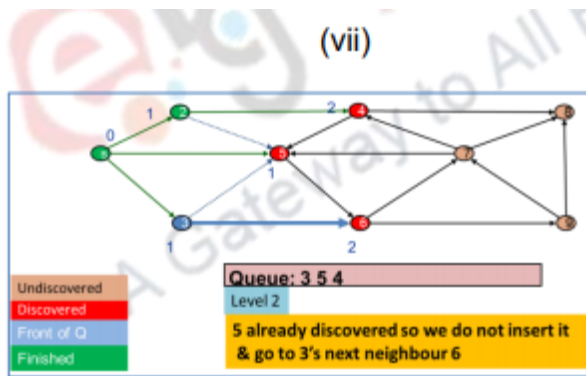


(v)



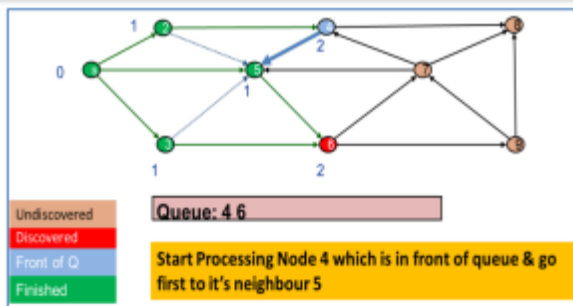
(vi)



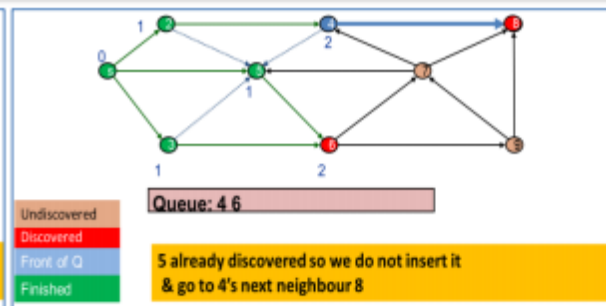


(xi)

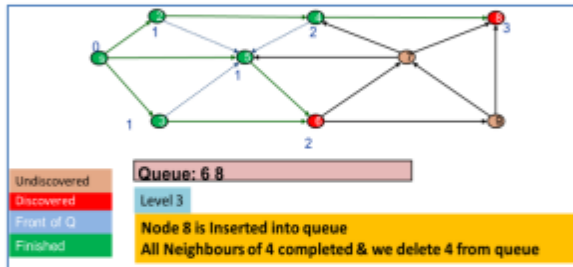
(xii)



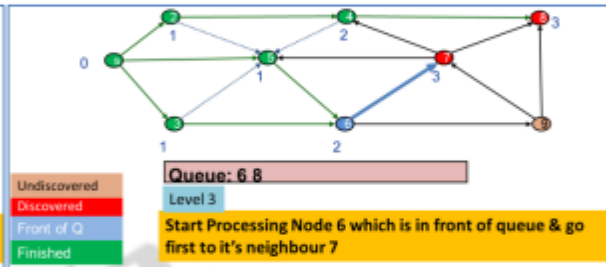
(xiii)



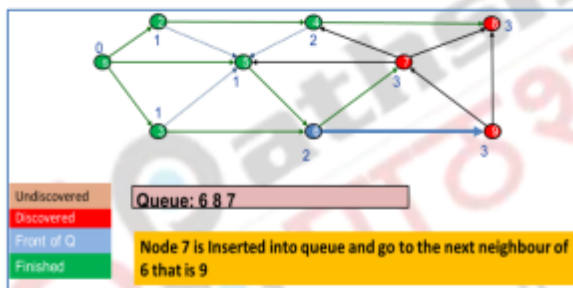
(xiv)



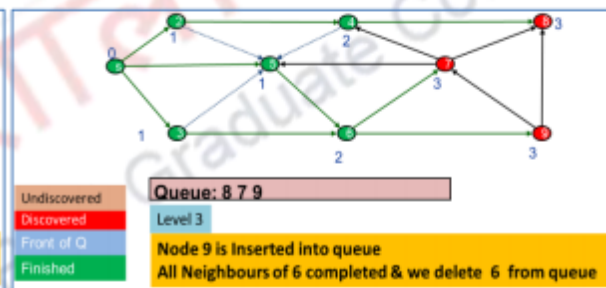
(xv)



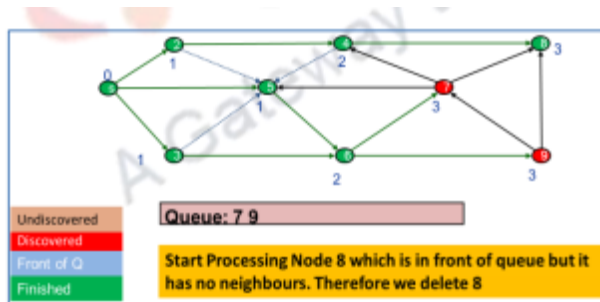
(xvi)



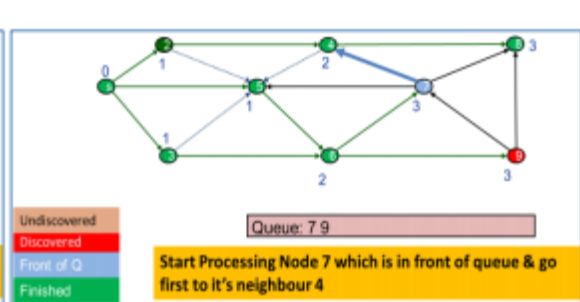
(xvii)



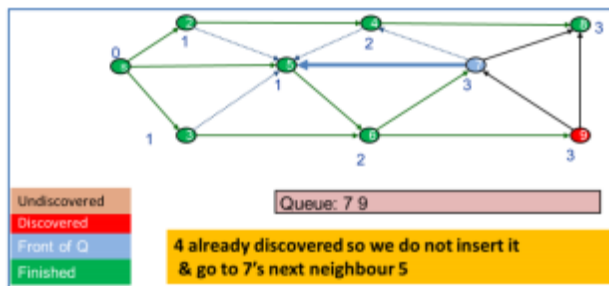
(xviii)



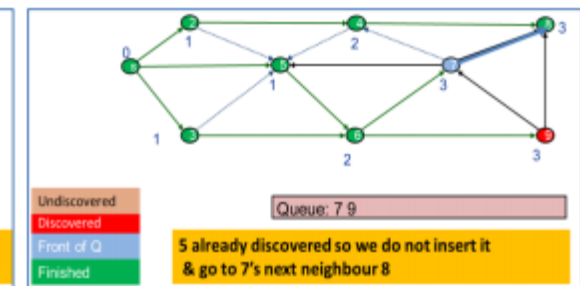
(xix)



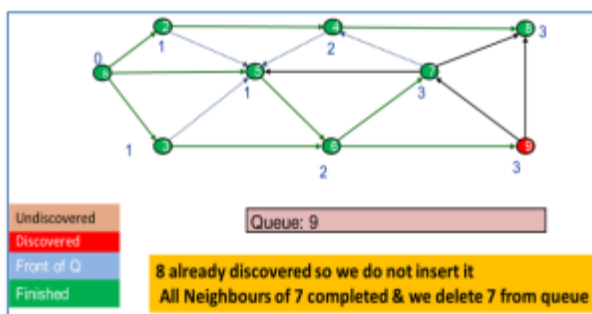
(xx)



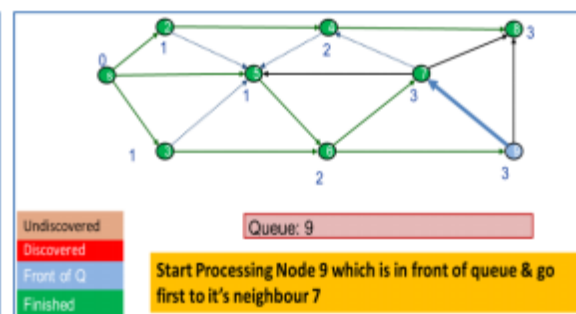
(xxi)



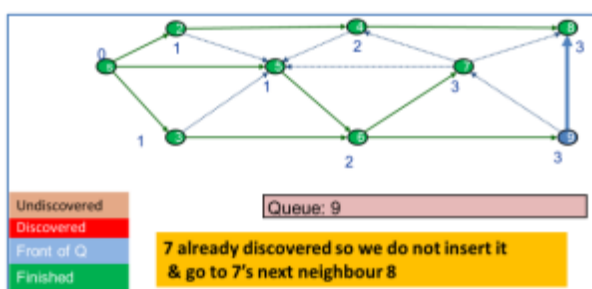
(xxii)



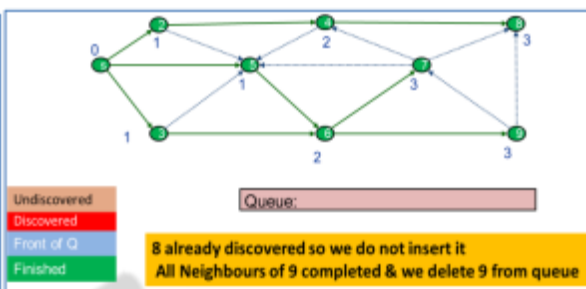
(xxiii)



(xxiv)



(xxv)



(xxvi)

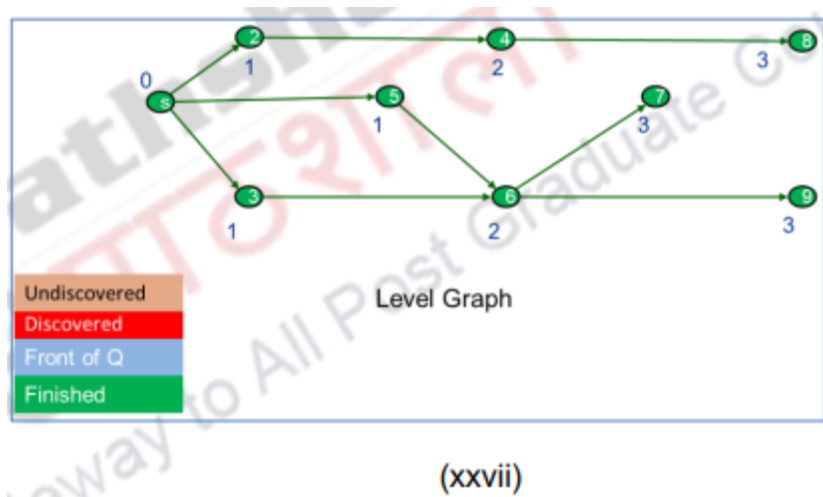


Figure 13.5 Running example for Breadth First Search using Queues

1. Figure 13.5 (i) shows the initially empty queue and the legend shows that each node can be in four states – Undiscovered, Discovered, Front of Queue or Finished. The directed graph has nodes s, 1, 2, 3, 4, 5, 6, 7, 8, and 9. Initially all nodes are undiscovered. The algorithm starts with the start node s and will finally find the goal node or will discover all the nodes either because goal node is not found or we want to traverse the complete graph in breadth first manner.

2. According to step 1 we **discover the start node s & enqueue s** Figure 13.5(ii). Now we are processing Level 0.

3. Now we go to Step 2 and enqueue all the neighbours of s which are 2, 3 and 5. We are processing at level 1. When all neighbours are enqueued we dequeue node s. (Figure 13.5 (iii-v)).

4. Now we start processing node 2 which is the node in front of the queue and start processing the neighbours of 2 (Figure 13.5 (vi)). Now we are processing at level 2.

5. Now we go to Step 3 and enqueue all the neighbours of 2 (4, 5) which have not yet been discovered (only 4 since 5 has already been discovered). When all undiscovered neighbours are enqueued, we dequeue node 2. (Figure 13.5 (vii)).

6. Now we start processing node 3 which is the node in front of the queue and start processing the neighbours of 3 (Figure 13.5 (viii)). We are still processing at level 2.

7. Now we go to Step 3 and enqueue all the neighbours of 3 (5, 6) which have not yet been discovered (only 6 since 5 has already been discovered) (Figure 13.5 (ix)).

When all undiscovered neighbours are enqueued, we dequeue node 3. (Figure 13.5 (x)).

8. Now we start processing node 5 which is the node in front of the queue and start processing the neighbours of 3 (Figure 13.5 (xi)). We are still processing at level 2.

9. Now we go to Step 3 and enqueue all the neighbours of 5 (only 6) which have not yet been discovered (no neighbour that has not been discovered) (Figure 13.5 xi)). When all undiscovered neighbours are enqueued, we dequeue node 5. (Figure 13.5 (xii)).

10. Now we start processing node 4 which is the node in front of the queue and start processing the neighbours of 4 (Figure 13.5 (xiii)). We are still processing at level 2.

11. Now we go to Step 3 and enqueue all the neighbours of 4 (5,8) which have not yet been discovered (only 8 since 5 has already been discovered)) (Figure 13.5 (xiv)). When all undiscovered neighbours are enqueued, we dequeue node 4. (Figure 13.5 (xv)). We are now processing at level 3

12. Now we start processing node 6 which is the node in front of the queue and start processing the neighbours of 6 (Figure 13.5 (xvi)). We are still processing at level 3.

13. Now we go to Step 3 and enqueue all the neighbours of 6 (7,9) which have not yet been discovered (7 & 9)) (Figure 13.5 (xvii)). When all neighbours are enqueued, we dequeue node 6. (Figure 13.5 (xviii)). We are now processing at level 3

14. Now we start processing node 8 which is the node in front of the queue and start processing the neighbours of 8 that are to be inserted. 8 has no neighbours (Figure 13.5 (xix)). We are still processing at level 3.

15. Now we start processing node 7 which is the node in front of the queue and start processing the neighbours of 7 (Figure 13.5 (xx)). We are still processing at level 3.

16. Now we go to Step 3 and enqueue all the neighbours of 7 (4,5,8) which have not yet been discovered (no neighbour that has not been discovered) -(Figure 13.5 (xxi-xxii)). Therefore we dequeue node 7. (Figure 13.5 (xxiii)). We are now processing at level 3

17. Now we start processing node 9 which is the node in front of the queue and start processing the neighbours of 9 (Figure 13.5 (xxiv)). We are still processing at level 3.

18. Now we go to Step 3 and enqueue all the neighbours of 9 (7,8) which have not yet been discovered (no neighbour that has not been discovered) -(Figure 13.5 (xxv-

xxvi)). Therefore we dequeue node 9. (Figure 13.5 (xxvi)). We are now processing at level 3

19. Now all nodes have been traversed and the algorithm is completed according to step 2.

As you can see the queue is the crucial data structure used in this algorithm for keeping track of nodes that are to be discovered.

Using Queues: Coding Messages

The queue is also used for encoding. In this example the coding is based on shifting a letter depending on where the letter is in the message. Here we use a *repeating key*, a sequence of integers to determine how much each character is to be shifted

Example: Consider the repeating key 3 1 7 4 2 5. The first character in the message is shifted by 3, the next by 1, the next by 7, and so on. The encoding and decoding using the repeating key is shown in Figure 13.6. When the key is exhausted, start over from beginning of the key. We can use a queue to store the values of the key, *dequeue* a key value when needed. After using it, *enqueue* it back onto the end of the queue so that the queue represents the constantly cycling values in the key. Note that there are *two* copies of the key, stored in two separate queues where the encoder has one copy and the decoder has a separate copy



Figure 13.6 Encoded Message Using a Repeating Key

Summary

- Discussed the different applications of Queue
- Explained the use of queue for printing jobs & customer service
- Explained through simulation the use of queues for Breadth First Search
- Discussed the use of queue for encoding messages using repeating key.

you can view video on Applications of Queue ADT



Web Links
www.site.uottawa.ca/~nelkadri/CSI%201101/LectureXI-Queues.ppt
www.cs.nyu.edu/~melamed/courses/102/lectures/Queues.ppt
ftp.csd.uwo.ca/courses/CS1027b/notes/CS1027-009-Queues-W12.ppt
www.cs.princeton.edu/~wayne/cs423/lectures/demo-bfs.ppt
Supporting & Reference Materials
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Robert Sedgewick, Kevin Wayne " Algorithms ", Addison-Wesley Professional, 4th Edition, 2011
Thomas H. Cormen, et.al, "Introduction to Algorithms" The MIT Press; 3rd edition, 2009
Anany Levitin "Introduction to the Design and Analysis of Algorithms", Pearson; 3rd edition 2011